

操作系统第八周星期三讲稿

💡 课前提醒：这节课讲经典进程同步互斥问题，这些问题是基础，一定要举一反三。考试的时候PV操作很容易出错，大家要特别注意。

一、引入：从机制到问题

前面我们讲的都是**机制、原理**：

- 纯软件方法（Peterson算法）
- 硬件辅助方法（TS指令）
- 信号量机制

这节课我们主要讲**实际问题**，这些都是非常经典的问题。

🎯 学习目标：根据这些经典问题，做到举一反三。把稍微复杂的问题拆解、变成这些经典问题或它们的组合，从而能够解决多进程并发的同步互斥问题。

二、信号量机制复习 ★

2.1 信号量的提出

- 1965年由 **Dijkstra** 提出
- P、V分别是荷兰语的 test(proberen) 和 increment(verhogen)
- 信号量只能通过**初始化和P、V两个标准原语**来访问
- 作为OS核心代码执行，不受进程调度的打断

2.2 计数信号量的数据结构

```
Type semaphore = record
    value : integer;    // 信号量值
    L : list of process; // 等待队列（PCB链表）
end
```

P操作（申请资源）：

```
Procedure P(S)
    S.value := S.value - 1;
    if S.value < 0 then
        block(S.L); // 阻塞，加入等待队列
    end
```

V操作（释放资源）：

```
Procedure V(S)
    S.value := S.value + 1;
    if S.value <= 0 then
        wakeup(S.L); // 唤醒等待队列中的一个进程
    end
```

2.3 信号量的物理意义 ★★

S.value 的值 含义

> 0	表示可用资源的个数
< 0	绝对值表示等待该资源的进程个数

- **P操作 (wait)**: 分配资源, 如果无法分配则阻塞
- **V操作 (signal)**: 释放资源, 如果有等待进程则唤醒

2.4 信号量的使用规则

1. ☒ 必须置一次且只能置一次初值
2. ☒ 只能由P、V操作来改变信号量的值
3. ☒ P、V操作本身是原子的 (关中断或TS指令实现)

三、信号量的两大基本应用

3.1 互斥

利用信号量实现进程互斥:

- **S = 1** (二元信号量/互斥信号量)

```
P(mutex);    // 进入临界区前
临界区
V(mutex);    // 退出临界区后
```

3.2 同步

利用信号量实现进程同步:

- **S = 0** (同步信号量, 表示某个条件是否满足)

例如: 描述进程执行的前驱/后继关系

```
// 进程A          // 进程B
...              P(S); // 等待A完成
V(S); // A完成后发信号 ...
```

四、经典问题一：生产者-消费者问题 ★★

4.1 问题描述

若干进程通过有限的共享缓冲区交换数据：

- **生产者**进程不断写入
- **消费者**进程不断读出
- 共享缓冲区共有 **N** 个
- 任何时刻只能有一个进程可对共享缓冲区进行操作

两个隐含条件：

1. 消费者和生产者数量不固定（可以有多个生产者、多个消费者）
2. 消费者和生产者不能同时使用缓冲区

4.2 行为分析

角色	行为	依赖条件
生产者	生产产品 → 放置产品	必须有空缓冲区
消费者	取出产品 → 消费产品	必须有产品

行为关系：

- 生产者之间：互斥（放置产品）
- 消费者之间：互斥（取出产品）
- 生产者与消费者之间：**互斥 + 同步**

4.3 信号量设置

```
semaphore mutex = 1; // 互斥访问缓冲区
semaphore empty = N; // 空闲缓冲区数量
semaphore full = 0;  // 产品数量
```

💡 恒等式：full + empty == N

4.4 PV操作代码

生产者：

```
P(empty);    // 申请空缓冲区
P(mutex);    // 互斥访问
one >> buffer // 放入产品
V(mutex);
V(full);     // 增加产品计数
```

消费者:

```
P(full);     // 申请产品
P(mutex);    // 互斥访问
one << buffer // 取出产品
V(mutex);
V(empty);    // 增加空缓冲区
```

4.5 ❌ 常见错误: P操作顺序问题

🔥 考点: 把mutex放在外面行不行?

```
// 错误写法!
P(mutex);    // 先拿互斥锁
P(empty);    // 再申请空缓冲区
...
```

问题分析:

- 如果缓冲区满了 (empty=0), 先来一个消费者
- 消费者 P(mutex) 成功进去了, 然后 P(full) 发现full=0, 阻塞
- 这时候生产者来了, 拿不到mutex, 也阻塞
- **死锁!** 系统卡住了, 谁也进展不了

✅ **结论:** 互斥信号量mutex的P操作一定要放在**最里面**, 临界区越小越好!

4.6 ✅ V操作顺序问题

释放资源的顺序一般来说**不会影响**, 因为:

- 你当前进程是活跃的
- 释放资源不会减少活跃程度
- 只会增加别人的活跃机会

所以两个V操作交换顺序没有问题。

4.7 完整伪代码

```
Semaphore full = 0;
Semaphore empty = n;
Semaphore mutex = 1;
ItemType buffer[0..n-1];
int in = 0, out = 0;
```

Producer:

```
producer() {
    while(true) {
        生产产品nextp;
        P(empty);
        P(mutex);
        buffer[in] = nextp;
        in = (in + 1) MOD n;
        V(mutex);
        V(full);
    }
}
```

Consumer:

```
consumer() {
    while(true) {
        P(full);
        P(mutex);
        nextc = buffer[out];
        out = (out + 1) MOD n;
        V(mutex);
        V(empty);
        消费nextc中的产品
    }
}
```

考试要求:

- 信号量命名要有意义，不要用ABCD
- 每个信号量的含义要写注释
- 写清楚PV操作的位置，我们主要判断你放的位置会不会引起死锁

五、用管程解决生产者-消费者问题

5.1 管程 vs 信号量

对比项	信号量	管程
互斥控制	显式声明mutex	管程 天然自带互斥锁 ，进入管程自动获得锁
同步控制	P、V操作分散在各个进程	条件变量（Condition Variable）封装在管程内部
代码可读性	P、V到处都是，容易乱	把同步互斥逻辑封装起来，更清晰

5.2 管程代码实现

```
monitor ProducerConsumer
    condition full, empty; // 条件变量，不是信号量！不含值
    integer count;

    procedure insert(item: integer);
    begin
        if count == N then wait(full); // 满了，等待full条件
        insert_item(item);
        count++;
        if count == 1 then signal(empty); // 从空变非空，发信号
    end;

    function remove: integer;
    begin
        if count == 0 then wait(empty); // 空了，等待empty条件
        remove = remove_item;
        count--;
        if count == N-1 then signal(full); // 从满变非满，发信号
    end;

    count := 0;
end monitor;
```

5.3 关键理解：条件变量与等待

❓ 思考题：当一个进程在管程中等待条件变量时，它是否还占用管程的互斥锁？

✅ 答案：必须释放！

如果一个进程wait在条件变量上还占着锁，那别的进程就进不了管程，条件永远得不到满足，就死锁了。

这就是管程最关键的设计——**wait条件变量时会自动释放管程锁**，被signal唤醒时又会重新获得锁。

使用方式：

```
procedure producer;
begin
    while true do
    begin
        item = produce_item;
```

```
        ProducerConsumer.insert(item); // 调用管程方法
    end
end

procedure consumer;
begin
    while true do
    begin
        item = ProducerConsumer.remove;
        consume_item(item);
    end
end
end
```

💡 **管程的本质**：除了封装函数，最核心就是**条件变量**。通过等待条件变量可以退出管程，被signal又可以重新激活。这大大减少了程序员自己加锁出错的可能性。

六、经典问题二：读者-写者问题 ★★

6.1 问题描述

对共享资源的读写操作，任一时刻：

- **读-写 互斥**
- **写-写 互斥**
- **读-读 允许** ✨（这是关键！多个读者可以同时读）

生活实例：12306订票系统

- 读者：查询余票（可以很多人同时查）
- 写者：订票/退票（一次只能一个人操作）

6.2 基本思路

Writer	Reader
P(wmutex);	if first_reader? then P(wmutex);
写数据	读数据
V(wmutex);	if last_reader? then V(wmutex);

核心思想：

- 第一个读者负责拿写锁，阻止写者进来
- 最后一个读者负责释放写锁，允许写者进来
- 中间的读者不用管锁，直接读就行

6.3 信号量与变量设置

信号量/变量	含义	初值
<code>wmutex</code>	表示"允许写"的互斥锁	1
<code>readcount</code>	表示"正在读"的进程数	0
<code>mutex</code>	保护readcount的互斥操作	1

6.4 读者优先算法

Writer	Reader
<code>P(wmutex);</code>	<code>P(mutex);</code>
<code>write</code>	<code>if readcount=0 then P(wmutex);</code>
<code>V(wmutex);</code>	<code>readcount := readcount + 1;</code>
	<code>V(mutex);</code>
	<code>read</code>
	<code>P(mutex);</code>
	<code>readcount := readcount - 1;</code>
	<code>if readcount=0 then V(wmutex);</code>
	<code>V(mutex);</code>

 **分析：**这个算法对谁有利？

只要有一个读者在读，后续来的读者都可以直接进来，写者必须等**所有**读者都读完才能写。

如果读者源源不断地来，写者可能**永远写不了** → **饿死**！

结论：**读者优先**（对读者有利）

6.5 写者优先的改进

需求：写者出现后，新读者不允许抢在该写者之前进行读操作。

实现思路：增加一个互斥信号量 `rwmutex`，让写者到来时把读者挡在外面。

6.6 读写公平的算法

增加信号量 `rwmutex` 控制读写公平：

Reader	Writer
<code>P(rwmutex);</code>	<code>P(rwmutex);</code>
<code>P(mutex);</code>	<code>P(wmutex);</code>
<code>if readcount=0 then P(wmutex);</code>	写数据
<code>readcount := readcount + 1;</code>	<code>V(wmutex);</code>

Reader	Writer
<code>V(mutex);</code>	<code>V(rwmutex);</code>
<code>V(rwmutex); // 注意这个位置!</code>	
<code>read</code>	
<code>P(mutex);</code>	
<code>readcount := readcount - 1;</code>	
<code>if readcount=0 then V(wmutex);</code>	
<code>V(mutex);</code>	

? 思考题: `V(rwmutex)` 能不能放在`read`之后?

如果放在`read`之后, 读者拿着`rwmutex`锁去读 (读可能很慢), 写者就被挡在外面, 本质又变成读者优先了。

放在`read`之前释放, 写者就有机会在读者读的过程中抢到`rwmutex`, 把后面的读者挡住。

七、经典问题三：哲学家进餐问题 ★★

7.1 问题描述

(由Dijkstra首先提出, Tony Hoare重新表述)

5个哲学家围绕一张圆桌而坐, 桌子上放着5支筷子, 每两个哲学家之间放一支。

哲学家的动作:

- **思考**: 不需要筷子
- **进餐**: 需要同时拿起他左边和右边的**两支**筷子

问题: 如何保证哲学家们的动作正确有序进行?

1. 不出现相邻者同时进餐
2. 不出现有人永远拿不到足够筷子 (死锁/饿死)

7.2 ✖ 简单但有问题的解法

```
Var chopstick : array[0..4] of semaphore; // 初值均为1

Philosopher i:
    P(chopstick[i]);           // 拿左边筷子
    P(chopstick[(i+1) mod 5]); // 拿右边筷子
    eat                        // 进餐
    V(chopstick[i]);
    V(chopstick[(i+1) mod 5]);
    think                      // 思考
```

问题：如果5个哲学家同时拿起左边筷子 → 所有人都拿不到右边筷子 → 死锁！

7.3 三种解决方案

方案	思路	实现方式
方案1	至多只允许4个哲学家同时尝试拿筷子	设置信号量 <code>room = 4</code> ，进餐厅前先P(room)
方案2	对拿筷子的顺序做规定	奇数号先拿左，再拿右；偶数号先拿右，再拿左
方案3	同时拿起两根筷子，否则不拿	AND信号量集 SP操作

7.4 方案3：基于AND信号量集的方法

```
Var chopstick : array[0..4] of semaphore;

Philosopher i:
    think                                // 思考
    SP(chopstick[(i+1) mod 5], chopstick[i]); // 同时申请两根筷子
    eat                                  // 进餐
    SV(chopstick[(i+1) mod 5], chopstick[i]); // 同时释放两根筷子
```

 AND信号量集的核心思想：要么全拿到，要么一个都不拿，避免部分占有。

八、扩展问题：仓库货物入库约束

问题描述

设有一个可以装A、B两种物品的仓库，容量无限大，但要求仓库中A、B两种物品的数量满足：

$$-M \leq A\text{物品数量} - B\text{物品数量} \leq N$$

即：

- A不能比B多N以上
- B不能比A多M以上

试用信号量和PV操作描述A、B两种物品的入库过程。

信号量设置

```
sa: A货物比B货物多的额度 // 初值=N
sb: B货物比A货物多的额度 // 初值=M

Semaphore mutex = 1, sa = N, sb = M;
```

算法描述

```
cobegin

procedure A:
  while(TRUE)
  begin
    P(sa);           // A入库会减少A-B的额度
    P(mutex);
    A产品入库;
    V(mutex);
    V(sb);           // A入库增加了B-A的额度
  end

procedure B:
  while(TRUE)
  begin
    P(sb);
    P(mutex);
    B产品入库;
    V(mutex);
    V(sa);
  end

coend
```

九、理发师问题

问题描述

理发店里有1位理发师、1把理发椅和n把供等候理发的顾客坐的椅子：

- 如果没有顾客，理发师便在理发椅上睡觉，当一个顾客到来时，叫醒理发师
- 如果理发师正在理发时，又有顾客来到，则如果有空椅子可坐，就坐下来等待，否则就离开

变种问题

1. **多理发师**：m位理发师，n把等候椅
2. **店满等待**：没有空椅子时不在门口等待，而不是直接离开

本讲总结

1. 信号量两大基本应用：

- 互斥：初值=1，临界区前后P、V
- 同步：初值=0，先做的后面V，后做的前面P

2. 三大经典问题：

- 生产者-消费者：empty/full + mutex，注意P操作顺序避免死锁
- 读者-写者：读者计数 + 写者互斥锁，可实现读者优先/写者优先/公平算法

- 哲学家进餐：AND信号量集避免死锁，或者限制人数/规定拿取顺序

3. 管程的本质：

- 天然互斥 + 条件变量
- wait时自动释放锁，避免死锁
- 把同步互斥逻辑封装，更不容易出错

4. 考试重点提醒：

- PV操作的顺序非常重要！mutex一定要放在最里面
- 信号量命名要有意义，写清楚注释
- 分析死锁可能性是高频考点

 最后忠告：同步互斥问题本身就很复杂，很容易出错。临界区一定要越小越好，占用公共资源要尽快释放！这样系统才公平、高效、不容易死锁。